PYTHON · OBJECT-ORIENTED PROGRAMMING

Understanding Inheritance: Reusing Code the Right Way

A practical guide comparing duplicated code with inheritance-based design, covering method resolution, real-world modeling, and the five types of inheritance supported by Python.

Code Duplication . Parent & Child Classes · super() · Method Resolution Order · Single & Multiple Inheritance · Hierarchical Inheritance · Hybrid Inheritance · Chess Piece Case Study

1. What Is Inheritance?

> Inheritance is a core principle of object-oriented programming. It allows one class — the child (or derived) class — to reuse the attributes and methods of another class, the parent (or base) class, instead of rewriting that logic from scratch.

To see why this matters, consider a simple example: modeling a `Dog` and a `Cat`, each with a name, an eating behavior, and a distinct sound.

1.1 The Problem: Writing Code Without Inheritance

dog_cat_normal.py

```python
class Dog:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print("Bark")
    def eat(self):
        print(f"{self.name} is eating")

class Cat:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print("Meow")
    def eat(self):
        print(f"{self.name} is eating")

dog = Dog("Tommy")
cat = Cat("Kitty")

dog.eat()
dog.speak()
cat.eat()
cat.speak()
```

OUTPUT

```
Tommy is eating
Bark
Kitty is eating
Meow
```

Both classes define an identical `__init__` constructor and an identical `eat()` method. The only real difference is the sound each animal makes. Now imagine extending this to a `Lion`, `Tiger`, `Horse`, and `Elephant` — the same constructor and `eat()` method would need to be copied into every new class.

> This is called code duplication. It means more lines of code, more places where a bug can hide, and more files to touch every time a shared behavior needs to change.

1.2 The Solution: Introducing a Parent Class

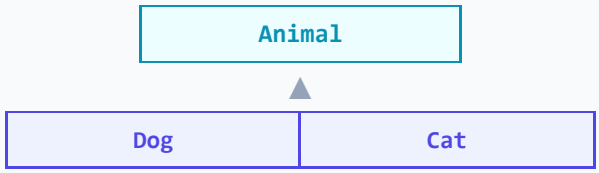
Instead of repeating the shared logic, it can be written once in a common parent class named `Animal`. Both `Dog` and `Cat` then inherit from it.

dog_cat_inheritance.py

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self):
        print(f"{self.name} is eating")

class Dog(Animal):
    def speak(self):
        print("Bark")

class Cat(Animal):
    def speak(self):
        print("Meow")

dog = Dog("Tommy")
cat = Cat("Kitty")

dog.eat()
dog.speak()
cat.eat()
cat.speak()
```

OUTPUT

```
Tommy is eating  ·  Bark  ·  Kitty is eating  ·  Meow  (identical to the version above)
```



Dog and Cat each inherit __init__() and eat() from Animal, and only define their own speak() method.

Only one copy of `eat()` now exists anywhere in the program. If its behavior needs to change, it is changed once, in `Animal`, and every subclass picks up the update automatically.

1.3 How Python Resolves Method Calls

When `dog.eat()` is called, Python looks for `eat()` first on the `Dog` class itself. It is not defined there, so Python walks up to the parent, `Animal`, finds it, and executes it. When `dog.speak()` is called, Python finds `speak()` directly on `Dog` and stops looking — it never needs to check `Animal` at all.

> Rule of thumb: Python always searches the object's own class first, then walks up the chain of parent classes until it finds a matching method or attribute.

2. A Second Example: Person, Student, and Teacher

The same pattern applies beyond animals. Consider a `Student` and a `Teacher`, both of whom have a name and a way to display it.

2.1 Without Inheritance

people_normal.py

```
class Student:
    def __init__(self, name): self.name = name
    def show(self): print(self.name)

class Teacher:
    def __init__(self, name): self.name = name
    def show(self): print(self.name)
```

Once again, the constructor and the `show()` method are duplicated word-for-word.

2.2 With Inheritance

people_inheritance.py

```
class Person:
    def __init__(self, name): self.name = name
    def show(self): print(self.name)

class Student(Person): pass
class Teacher(Person): pass

s = Student("Alice")
t = Teacher("Bob")
s.show(); t.show()
```

OUTPUT

```
Alice
Bob
```

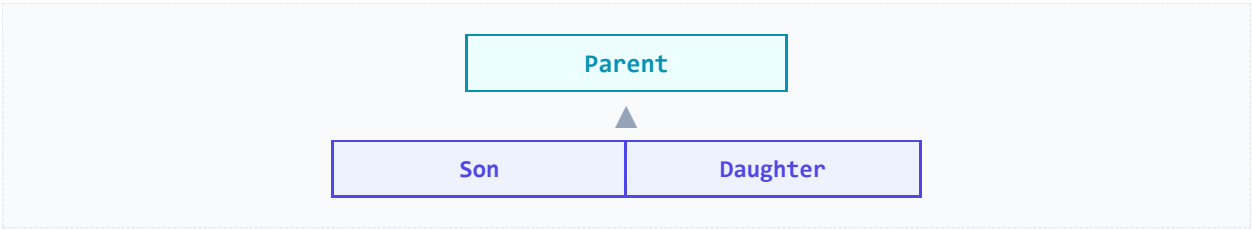
Notice that `Student` and `Teacher` contain no code of their own — just the keyword `pass`, which means *"this class doesn't add anything new yet."* They still work correctly because they inherit every method and attribute from `Person`.

3. Comparing the Two Approaches

Without Inheritance	With Inheritance
Same code written many times	Common code written once
More lines of code overall	Fewer lines of code
Harder to maintain as classes grow	Easier to maintain and extend
Changes must be repeated in every class	Change made once, in the parent class
Higher chance of inconsistent bugs	Lower chance of bugs

3.1 A Real-Life Analogy

Think of a family. If a parent has brown eyes and black hair, their children inherit those characteristics automatically — while still being free to develop their own, unique traits, such as a son who plays football or a daughter who plays piano.



The same idea applies in code: a child class automatically receives everything defined on its parent, and can still add behavior that makes it unique.

animal_bark.py

```
class Animal:
    def eat(self): print("Eating")

class Dog(Animal):
    def bark(self): print("Barking")
```

`Dog` inherits `eat()` from `Animal`, and additionally defines its own `bark()` method.

> Key takeaway: Inheritance lets a child class reuse the attributes and methods of a parent class. It reduces duplication, improves maintainability, and models real-world "is-a" relationships — for example, a Dog is an Animal.

4. Case Study: Modeling a Chess Set

Chess is one of the clearest real-world illustrations of inheritance, because every piece shares certain properties — a color, a reference to the chess set, a shape, and a move() behavior — while each piece moves in its own distinct way.

4.1 Without Inheritance

Modeling each piece independently forces the same constructor logic to be repeated across every class:

chess_normal.py

```
class Rook:
    def __init__(self, color, chess_set):
        self.color = color
        self.chess_set = chess_set
        self.shape = "♖"
    def move(self):
        print("Moves horizontally or vertically")

class Bishop:
    # ...identical __init__ pattern, different shape and move()

# ...the same pattern repeats for Knight, Queen, King, and Pawn
```

If a new shared attribute — say, position — needed to be added later, every single piece class (Rook, Bishop, Knight, Queen, King, Pawn) would have to be edited individually. That is a significant maintenance burden.

4.2 With Inheritance

A shared `Piece` base class removes the repetition. Each piece then calls `super().__init__()` to run the parent's constructor, before adding its own shape and movement logic.

chess_inheritance.py

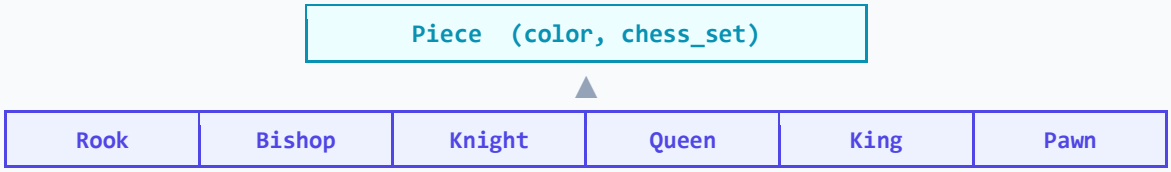
```
class Piece:
    def __init__(self, color, chess_set):
        self.color = color
        self.chess_set = chess_set

class Rook(Piece):
    def __init__(self, color, chess_set):
        super().__init__(color, chess_set)
        self.shape = "♖"
    def move(self): print("Moves horizontally or vertically")

class Queen(Piece):
    def __init__(self, color, chess_set):
        super().__init__(color, chess_set)
        self.shape = "♕"
    def move(self): print("Moves horizontally, vertically and diagonally")

rook = Rook("White", "Chess Game")
queen = Queen("Black", "Chess Game")
rook.move(); queen.move()
```

OUTPUT

```
White
Black
Moves horizontally or vertically
Moves horizontally, vertically and diagonally
```

What does super() do?

The call `super().__init__(color, chess_set)` invokes the parent class's constructor directly. Instead of writing `self.color = color` and `self.chess_set = chess_set` in every subclass, Python executes `Piece.__init__()` on behalf of the developer.



Each child class inherits the shared state from Piece and implements its own shape and move().

4.3 Extending the Base Class Further

Common *behavior*, not just shared data, can also live in the parent class. Adding a `display_info()` method to `Piece` immediately makes it available to every piece in the hierarchy:

chess_display_info.py

```
class Piece:
    def __init__(self, color, chess_set):
        self.color = color; self.chess_set = chess_set
    def display_info(self):
        print(f"{self.color} {self.__class__.__name__}")

# rook.display_info()   -> "White Rook"
# queen.display_info()  -> "Black Queen"
```

> Design principle: Put everything that is common into the parent class, and let each child class define only what makes it different. In the chess model, Piece holds the shared state, while Rook, Bishop, Knight, Queen, King, and Pawn each provide their own movement rules.

5. The Five Types of Inheritance in Python

Python supports five structural patterns of inheritance. Each solves a different modeling problem.

Type	Structure	Typical Example
Single	A → B	Animal → Dog
Multiple	A, B → C	Flyer + Swimmer → Duck
Multilevel	A → B → C	Animal → Mammal → Dog
Hierarchical	A → B, C, D	Animal → Dog, Cat, Horse
Hybrid	Combination of the above	Person → Student & Employee → TeachingAssistant

1 · SINGLE INHERITANCE

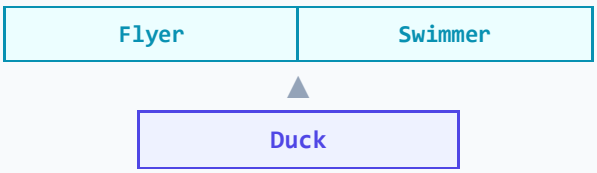
One parent class, one child class.

single.py

```
class Animal:
    def eat(self): print("Animal is eating")
class Dog(Animal):
    def bark(self): print("Dog is barking")
dog = Dog()
dog.eat()   # inherited
dog.bark()  # own method
```

2 · MULTIPLE INHERITANCE

One child class inherits from more than one parent class at the same time.



multiple.py

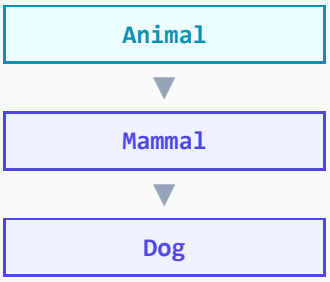
```
class Flyer:
    def fly(self): print("Can fly")
class Swimmer:
    def swim(self): print("Can swim")
class Duck(Flyer, Swimmer): pass
duck = Duck()
duck.fly(); duck.swim()
```

OUTPUT

```
Can fly
Can swim
```

3 · MULTILEVEL INHERITANCE

Inheritance occurs across a chain of multiple levels.



multilevel.py

```
class Animal:
    def eat(self): print("Eating")
class Mammal(Animal):
    def walk(self): print("Walking")
class Dog(Mammal):
    def bark(self): print("Barking")
dog = Dog()
dog.eat(); dog.walk(); dog.bark()
```

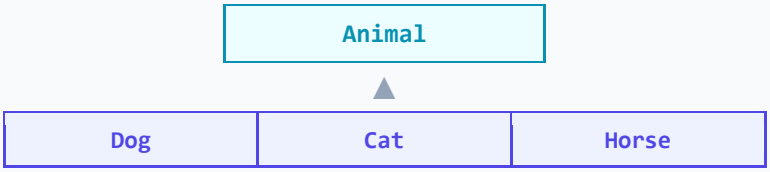
OUTPUT

```
Eating
Walking
Barking
```

`Dog` inherits `walk()` from `Mammal`, and `eat()` from `Animal`, two levels up.

4 · HIERARCHICAL INHERITANCE

One parent class has multiple, independent child classes.



hierarchical.py

```
class Animal:
    def eat(self): print("Eating")
class Dog(Animal): def bark(self): print("Bark")
class Cat(Animal): def meow(self): print("Meow")
```

6. Hybrid Inheritance in Depth

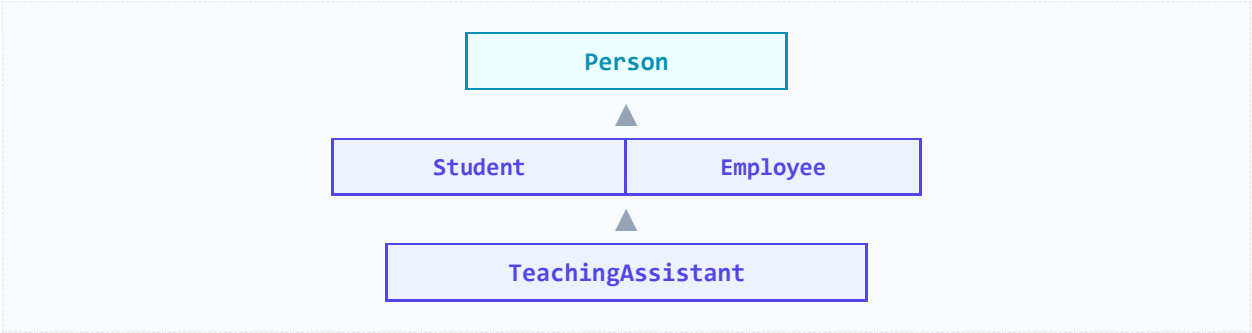
Hybrid inheritance combines two or more of the patterns above. It is common in real-world systems where an entity naturally belongs to more than one category.

6.1 Case Study: A University System

Consider a university membership system with the following rules:

- Every **Person** has a name.
- A **Student** is a Person.
- An **Employee** is also a Person.
- A **TeachingAssistant** is both a Student and an Employee.

This combines **hierarchical inheritance** (Person → Student, Employee) with **multiple inheritance** (TeachingAssistant → Student + Employee), producing hybrid inheritance overall.



university_hybrid.py

```
class Person:
    def __init__(self, name): self.name = name
    def show_name(self): print("Name:", self.name)

class Student(Person):
    def study(self): print(self.name, "is studying")

class Employee(Person):
    def work(self): print(self.name, "is working")

class TeachingAssistant(Student, Employee):
    def assist(self): print(self.name, "is assisting the professor")

ta = TeachingAssistant("Alice")
ta.show_name(); ta.study(); ta.work(); ta.assist()
```

OUTPUT

```
Name: Alice
Alice is studying
Alice is working
Alice is assisting the professor
```

6.2 Why This Matters

Without inheritance, a `TeachingAssistant` class would need to duplicate every method from both `Student` and `Employee`:

without_hybrid.py

```
class TeachingAssistant:
    # repeat all Student code
    # repeat all Employee code
```

With hybrid inheritance, one line accomplishes the same result:

with_hybrid.py

```
class TeachingAssistant(Student, Employee): pass
```

`TeachingAssistant` automatically receives `show_name()` from `Person`, `study()` from `Student`, and `work()` from `Employee` — while still being free to add its own methods, such as `assist()`.

> Note on conflicts: When two parent classes define a method with the same name, Python resolves the ambiguity using the Method Resolution Order (MRO) — a deterministic left-to-right search order across the inheritance chain.

7. Summary

Inheritance is the mechanism that allows a class to reuse the attributes and methods of another class. It removes duplicated logic, centralizes shared behavior in a single place, and models natural real-world hierarchies. Python supports five structural patterns — single, multiple, multilevel, hierarchical, and hybrid — each suited to a different kind of relationship between classes.

Aspect	Without Inheritance	With Inheritance
Code volume	High — logic repeated per class	Low — shared logic written once
Maintenance	Every class edited individually	One edit in the parent class
Consistency	Prone to drift between copies	Guaranteed consistent behavior
Extensibility	New classes duplicate everything again	New classes simply subclass the parent

> Guiding principle: place everything that multiple classes have in common into a shared parent class, and let each child class define only what makes it unique.